

GitHub Copilot 補完活用ガイド

AI ワークフローで人が書くコードを補助する

(株) SRA

Version 0.1.0

2026-05-29

GitHub Copilot 補完活用ガイド

この文書は、AI を用いたアプリ生成ワークフローの中で、GitHub Copilot の補完をどこに使うかを整理する説明書である。

このワークフローでは、AI が仕様化前ノート、三点セット、実装プロンプト、テスト観点などの中間成果物を作る。ただし、レビューへの回答、最終判断、AI が生成しない部分のコード、細部の実装修正は人が行う。

GitHub Copilot は、その人が行う実装作業を補助する道具として使う。Copilot に仕様判断を任せるのではなく、人が決めた内容を既存コードの文脈に沿って素早くコード化するために使う。

1. 全体像

AI ワークフローと GitHub Copilot の役割は分けて考える。

役割	主な担当	内容
仕様化	AI と人	要求を整理し、仕様化前ノートと三点セットを作る
レビュー	AI と人	矛盾、未決事項、推測、実装漏れを洗い出す
判断	人	仕様の採否、レビュー指摘への回答、業務判断を行う
実装補完	Copilot と人	人が決めた修正や不足コードを補完する
確認	AI と人	テスト、レビュー、仕様との照合を行う

基本の流れは次の通りである。

仕様化前ノート

-> 三点セット

-> 実装プロンプト

-> AI 生成コード

-> レビュー

-> 人が修正方針を決める

-> Copilot で補完しながら実装する

-> テストと再レビューで確認する

Copilot を使うのは、主に「人が修正方針を決めた後」である。

2. 基本方針

Copilot は、仕様を決める道具ではない。既に決まっている仕様、レビュー回答、テスト観点、既存コードのパターンをもとに、コードを書く作業を補助する道具である。

次の方針で使う。

- 仕様判断は人が行う。
- レビュー指摘の採否は人が決める。
- Copilot の補完は候補として扱う。
- 補完されたコードは必ず読む。
- 仕様、三点セット、テストと照合する。
- 補完が大きくなりすぎたら、小さく分けて確認する。

Copilot は、入力中のファイル、周辺コード、コメント、関数名、テスト名から文脈を推測する。そのため、補完前に人が意図を短くコメントや関数名で示すと、期待に近い補完になりやすい。

3. レビュー指摘への対応で使う

レビューで見つかった指摘に対し、人が修正方針を決めた後、Copilot で実装修正を補助する。

指摘の種類	Copilot を使う作業	人が確認すること
境界値が抜けている	下限、上限、0、空値のチェックを追加する	条件が仕様と一致しているか
入力検証が足りない	バリデーション関数やエラー表示を追加する	エラー条件と表示文言が正しいか
状態遷移が足りない	ボタン状態、画面状態、内部状態の更新を追加する	状態の戻し忘れがないか
エラー処理が弱い	例外処理、戻り値チェック、ログ出力を追加する	エラーを握りつぶしていないか
テストが不足している	既存形式に合わせてテストケースを追加する	本当に指摘を検出できるか

レビューへの回答では、Copilot に修正の意味まで判断させない。人が「この条件を追加する」「この挙動に変更する」と決めてから補完を使う。

4. AI が生成しない部分を書く

AI 生成コードには、未実装箇所、TODO、プロジェクト固有の接続処理が残ることがある。これらは人が周辺コードを確認しながら実装する。

Copilot が向いている作業は次である。

- 未実装関数の本体を書く。

- TODO 箇所を既存パターンに合わせて埋める。
- Qt の signal/slot 接続を書く。
- UI 部品とデータモデルを接続する。
- 画面遷移後の状態更新を書く。
- 保存、読込、初期化などの定型処理を書く。

ただし、TODO の意味が仕様上未決である場合は、補完してはいけない。まず仕様化前ノートや三点セットに戻り、未決事項として扱う。

5. 定型的な UI 実装で使う

Qt などの UI 実装では、定型的なコードが多い。Copilot は、既存コードの書き方に合わせて、こうした部分を補完しやすい。

対象	補完に向く内容
Widget 初期化	入力欄、ボタン、ラベル、テーブルの初期設定
レイアウト	既存レイアウトに合わせた部品追加
表示更新	モデルの値を画面へ反映する処理
入力取得	UI 部品から値を読み取る処理
有効無効制御	状態に応じた enable / disable 切り替え
メッセージ表示	入力エラー、処理完了、確認表示

UI 実装では、見た目だけでなく操作結果が仕様と合っているかを確認する。補完で画面部品が増えた場合は、UI仕様書にない部品や操作が混ざっていないかを見る。

6. ビジネスルールの実装補助で使う

計算、検証、状態変更などのビジネスルールは、仕様が確定している場合に限って Copilot を使う。

向いている作業は次である。

- 確定済みの計算式をコード化する。
- 上限、下限、丸め、桁数制約を実装する。
- 入力エラー条件を列挙する。
- 結果オブジェクトを組み立てる。
- 既存の Validator や Service の形に合わせる。

ビジネスルールでは、Copilot の補完がそれらしく見えても誤ることがある。特に、丸め、端数処理、負数、0件、空値、日付境界、通貨、単位変換は人が必ず確認する。

7. テストコードで使う

Copilot は、既存テストの形式に合わせてテストケースを増やす用途に向いている。三点セットがあると、テストの分類が明確になる。

三点セット	テストの分類	Copilot を使う作業
UC仕様書	シナリオテスト	正常系、異常系、代表フローのテストを追加する
UI仕様書	UIテスト、手動確認	入力、表示、ボタン状態の確認コードを追加する
ビジネスレイヤー仕様書	ロジックテスト	境界値、計算、検証、状態変更のテストを追加する

テストコードでは、次の補完が有効である。

- 正常系テストの雛形を作る。
- 異常系テストを追加する。
- 境界値テストをパラメータ化する。
- モックやスタブを既存形式に合わせて作る。
- レビュー指摘に対応する回帰テストを追加する。

補完されたテストは、テスト名、入力値、期待値を人が確認する。テストが仕様と違う期待値を固定してしまうと、バグを守るテストになる。

8. リファクタリングで使う

レビュー後の修正で、コードを少し整理した方がよい場合がある。Copilot は、既存パターンに合わせた機械的なリファクタリング補助に使える。

使いやすい作業は次である。

- 長い関数を小さな関数に分ける。
- 重複した条件判定を共通関数にする。
- 既存の命名規則に合わせて変数名を揃える。
- 類似クラスの形式に合わせて新しいクラスを追加する。
- include や import を補完する。

リファクタリングでは、仕様変更が混ざりやすい。Copilot が挙動を変える補完を出した場合は採用しない。変更前後でテストが同じ意味を確認しているかを見る。

9. ドキュメント作成で使う

Copilot は、コード周辺の短い説明を補完する用途にも使える。

使いやすい作業は次である。

- 関数コメントを書く。
- レビュー対応メモを書く。
- テストケースの意図を書く。
- 制約条件を短く補足する。
- README や開発メモに追記する。

ドキュメント補完でも、仕様判断を Copilot に任せない。補完された説明が仕様書や実装と一致しているかを確認する。

10. Copilot に任せないこと

次の作業は Copilot に任せない。

任せないこと	理由
仕様判断	周辺コードから推測して、それらしい仕様を作る可能性がある
レビュー指摘の採否	指摘を受け入れるかどうかは人の判断である
未決事項の解決	未決事項を補完で埋めると、仮決めが仕様化される
セキュリティ判断	安全性の判断には文脈と責任が必要である
業務ルールの解釈	業務知識はコードだけでは決まらない
テスト十分性の判断	テストが十分かどうかは仕様とリスクから判断する

Copilot は強力な補完であるが、判断者ではない。補完候補を採用する責任は人が持つ。

11. 推奨する使い方

実作業では、次の順で使う。

1. 三点セットとレビュー指摘を読む。
2. 人が修正方針を決める。
3. 変更箇所を小さく決める。
4. 関数名、コメント、テスト名で意図を明示する。
5. Copilot の補完を受ける。
6. 補完結果を読む。

7. 仕様、テスト、レビュー指摘と照合する。
8. 必要なら AI に差分レビューをさせる。
9. テストを実行する。

Copilot の補完が大きくなりすぎる場合は、関数単位、条件単位、テストケース単位に分ける。小さく補完し、小さく確認する方が、間違いを見つけやすい。

12. よくある失敗

失敗	起きること	対策
補完結果をそのまま採用する	仕様外の条件や処理が混ざる	必ず仕様と照合する
仕様未確定のまま補完する	未決事項がコードに埋め込まれる	先に未決事項を解決する
大きな補完を一度に採用する	レビューしにくくなる	小さな単位で補完する
テストなしで修正する	変更の正しさを確認できない	回帰テストを追加する
Copilot に業務判断をさせる	もっともらしい誤実装になる	判断は人が行う

13. まとめ

GitHub Copilot は、AI ワークフローの中で人が書くコードを速くするための補完道具である。

仕様化前ノートや三点セットで何を作るかを整理し、レビューで何を直すかを決める。その後、人が決めた修正を Copilot で補完し、テストとレビューで確認する。

重要なのは、次の分担である。

AI は、仕様化、変換、レビュー観点の列挙を支援する。
人は、採用、判断、責任を持つ。
Copilot は、人が決めたことをコードに落とす作業を補完する。

この分担を守ると、Copilot の補完能力を活かしながら、仕様の仮決めやレビュー漏れを減らしやすくなる。